

Confidentiality?

What This Lab Is About

This lab is designed to test and strengthen your understanding of **core cryptography ideas** introduced in:

- **Lecture 1:** Fundamentals of Computer Security
- **Lecture 2:** Foundations of Cryptography

In Lecture 1, we discussed that one of the fundamental goals of computer security is **confidentiality**, i.e., keeping non-public information accessible only to authorized parties, and that **encryption** is one of the key tools used to support confidentiality.

In Lecture 2, we introduced the basic model of encryption and decryption:

- Encryption transforms **plaintext** into **ciphertext** to provide confidentiality
- Decryption reverses that transformation using a key

We also defined the key terms *plaintext*, *ciphertext*, and *encryption/decryption keys*.

This lab uses small, readable examples so you can practice recognizing transformations and reasoning about why some approaches are weak.

Learning Objectives

By completing this lab, you should be able to:

1. **Explain confidentiality and why cryptography helps**
 - Connect confidentiality (Lecture 1) to encryption as a protection mechanism.
2. **Use the plaintext/ciphertext model correctly**
 - Identify what is plaintext vs. ciphertext, and what “recovering plaintext” means in practice.

3. Reason about security of a cipher using key space

- Understand why small key spaces make brute-force (exhaustive key search) feasible.

4. Adopt an attacker's viewpoint

- Practice “ciphertext-only” style thinking: given only transformed text, how could the original be recovered?

Challenge 1: Multi-step Transformations

Scenario

You are given a text file (`output.txt`) containing **five lines**. Each line is produced by applying a different transformation to **one part** of a hidden flag.

Your Task

1. Identify what kind of transformation each line uses.
2. Reverse each transformation.
3. Reconstruct the original flag by putting the pieces together in order.

Takeaways

- Recognizing the difference between encryption-like transformations and encodings/representations.
- How small key spaces make exhaustive key search feasible.
- Why certain simple transformations do not provide real confidentiality.

Questions to Consider

- Which lines were actually encrypted, and which were just encoded?
- How does key space size affect the difficulty of reversing a transformation?
- Why does reversing these transformations not require breaking strong cryptography?

Challenge 2: One-Time Pad Misuse

Scenario

You are given a file called `otp_challenge.txt` containing a known plaintext, its ciphertext, and a second ciphertext encrypting a secret flag. Both ciphertexts use the same XOR-based pad (key).

Your Task

1. Use the known plaintext and ciphertext to recover the pad.
2. Apply the pad to the second ciphertext.
3. Recover the hidden flag from the result.

Takeaways

- OTP reuse breaks confidentiality even though OTP is theoretically secure.
- XOR properties can be exploited in known-plaintext attacks.
- The importance of correct cryptographic usage over algorithm choice.

Questions to Consider

- Why does XORing two ciphertexts eliminate the key in OTP reuse?
- What conditions are required for OTP to be perfectly secure?
- How does a known-plaintext attack enable flag recovery in this scenario?

Challenge 3: The Penguin

Scenario

You are given a file called `flag.dat` containing encrypted and encoded data. The file preserves some structure from the original plaintext.

Your Task

1. Determine how the data was encoded or transformed.
2. Reverse the transformation of the data.
3. Discover the original file format.
4. Recover the hidden flag.

Takeaways

- Encryption that handles blocks independently can leak structural information.
- Patterns in data may survive encryption, highlighting practical confidentiality limitations.
- Encoding alone does not provide security; encryption must be applied carefully.

Questions to Consider

- Why can independent block processing leak structure?
- What type of information can an attacker infer without decrypting?
- Why is encoding not equivalent to encryption?

Challenge 4: Modified Vigenère Cipher

Scenario

You are given a ciphertext produced using a modified Vigenère cipher. The key length is 4, with only the last key character unknown. Plaintext is all uppercase letters.

Your Task

1. Use the known key length and known key characters to narrow down possibilities.
2. Reason about the effect of the positional drift applied during encryption.
3. Recover the unknown key characters.
4. Decrypt the ciphertext to recover the original plaintext.

Takeaways

- Short, partially known keys are vulnerable even with small modifications.
- Deterministic transformations do not prevent cryptanalysis.
- Classical ciphers fail against attackers with partial knowledge.

Questions to Consider

- How does partial knowledge of a key reduce the effective key space?
- Why does a short key make polyalphabetic ciphers weak?
- How can predictable positional modifications be exploited?

Challenge 5: The Generator of Keys

Scenario

You are given a compiled binary called **program** and a server using the same validation logic. You can submit strings to the server, which returns the flag only if the key is valid.

Your Task

1. Analyze the binary using reverse engineering tools such as **Binary Ninja** or **Ghidra**.
2. Identify constraints imposed on the key (such as length, character restrictions, or relationships between characters).
3. Use these constraints to significantly reduce the key search space.
4. Generate a key that satisfies all validation checks (e.g., via a simple key search or scripted generator).

Takeaways

- Custom cryptographic validation is risky and often insecure.
- Reverse engineering closed-source binaries can reveal critical information.
- Small or structured key spaces are exploitable.
- Security through obscurity is ineffective; real cryptography relies on well-analyzed primitives.

Questions to Consider

- Why is security through obscurity considered ineffective?
- How can reverse engineering reveal key validation rules?
- How does key space structure affect attack feasibility?
- Why is custom key validation generally discouraged in cryptography?